

Bioinformatics P 11

Name: Sidharthan S C

Roll Number: BE21B039

Question 1

[illegible]

```
In [9]: def propensity(seq,ss):
n = len(seq)
print(n)
AA_all = ['A','C','D','E','F','G','H','I','K','L','M','N','P','Q','R','S','T','V','W','Y']

composition = [0]*20 #Number of occurrence of each AA Acid.
H = [0]*20 #Number of occurrence of each AA in a helix.
ratio_H = [0]*20
propensity = [0]*20

for i in range(n):
    aa = seq[i]
    ind = AA_all.index(aa)
    composition[ind]+=1
    ty = ss[i]
    if str(ty)=='H':
        H[ind]+=1

print('Composition')
print(composition)
print('\nHelix composition')
print(H)
nH = ss.count('H')
print(nH)
percent_H = nH/n
print('\nPercentage Helix = %4.3f' %percent_H)

for i in range(20):
    ratio_H[i]=H[i]/composition[i]
    propensity[i] = ratio_H[i]/percent_H
print('\nPercentage AA in helix wrt overall')
print(ratio_H)
print('\nThe propensity of each AA is -')
for i in range(20):
    print('%s %4.3f' %(AA_all[i], propensity[i]))
print('\nMost preferred AA in helix - with cutoff = 1.000')
for i in range(20):
    if propensity[i]>=1.000:
        print('%s %4.3f' %(AA_all[i], propensity[i]))
```

```
In [10]: propensity(sequence,sec_str)

200
Composition
[25, 4, 5, 5, 20, 25, 7, 17, 1, 20, 4, 3, 9, 4, 9, 9, 8, 7, 11, 7]

Helix composition
[19, 2, 0, 1, 10, 13, 3, 10, 0, 12, 3, 2, 1, 0, 3, 6, 3, 2, 6, 2]
98

Percentage Helix = 0.490

Percentage AA in helix wrt overall
[0.76, 0.5, 0.0, 0.2, 0.5, 0.52, 0.42857142857, 0.5882352941176471, 0.0, 0.6, 0.75, 0.6666666666666666, 0.1111111111111111,
1, 0.0, 0.3333333333333333, 0.6666666666666666, 0.375, 0.2857142857142857, 0.5454545454545454, 0.2857142857142857]

The propensity of each AA is -
A 1.551
C 1.020
D 0.000
E 0.408
F 1.020
G 1.061
H 0.875
I 1.200
K 0.000
L 1.224
M 1.531
N 1.361
P 0.227
Q 0.000
R 0.680
S 1.361
T 0.765
V 0.583
W 1.113
Y 0.583

Most preferred AA in helix - with cutoff = 1.000
A 1.551
C 1.020
F 1.020
G 1.061
I 1.200
L 1.224
M 1.531
N 1.361
S 1.361
W 1.113
```

Question 2

Propensity

Length of total sequence = 200 [N]

Length of helix position in seq = 98 [N₂]

Amino Acid	Freq. in seq. entirely (N ₁)	Freq. in helix (N ₂)
A	25	19
R	9	3
N	3	2

$$P[A] = \frac{19/25}{98/200} = 1.55$$

$$P[R] = \frac{3/9}{98/200} = 0.67$$

$$P[N] = \frac{2/3}{98/200} = 1.31$$

Question 3

```

In [2]: def matchingString(x,y):
    match=''
    for i in range(0,len(x)):
        for j in range(0,len(y)):
            k=1
            # now applying while condition untill we find a substring match and length of substring is less than length of x and
            while (i+k <= len(x) and j+k <= len(y) and x[i:i+k]==y[j:j+k]):
                if len(match) <= len(x[i:i+k]):
                    match = x[i:i+k]
                    k=k+1
            return match
def secondary_str(seq):

    helix = {'A': 'Ha', 'C': 'Ia', 'D': 'Ia', 'E': 'Ha', 'F': 'ha',
             'G': 'Ba', 'H': 'ha', 'I': 'Ia', 'K': 'Ia', 'L': 'Ha',
             'M': 'ha', 'N': 'ba', 'P': 'Ba', 'Q': 'ha', 'R': 'Ia',
             'S': 'Ia', 'T': 'Ia', 'V': 'ha', 'W': 'ha', 'Y': 'ba'}

    strand = {'A': 'Ib', 'C': 'hb', 'D': 'Ib', 'E': 'Bb', 'F': 'hb',
              'G': 'Ib', 'H': 'bb', 'I': 'Hb', 'K': 'bb', 'L': 'hb',
              'M': 'Hb', 'N': 'bb', 'P': 'bb', 'Q': 'hb', 'R': 'Ib',
              'S': 'bb', 'T': 'hb', 'V': 'Hb', 'W': 'hb', 'Y': 'hb'}

    propensity_alpha = {'A': 1.45, 'C': 0.77, 'D': 0.98, 'E': 1.53, 'F': 1.12,
                        'G': 0.53, 'H': 1.24, 'I': 1.00, 'K': 1.07, 'L': 1.34,
                        'M': 1.20, 'N': 0.73, 'P': 0.59, 'Q': 1.17, 'R': 0.79,
                        'S': 0.79, 'T': 0.82, 'V': 1.14, 'W': 1.14, 'Y': 0.61}

    propensity_beta = {'A': 0.97, 'C': 1.30, 'D': 0.80, 'E': 0.26, 'F': 1.28,
                       'G': 0.81, 'H': 0.71, 'I': 1.60, 'K': 0.74, 'L': 1.22,
                       'M': 1.67, 'N': 0.65, 'P': 0.62, 'Q': 1.23, 'R': 0.90,
                       'S': 0.72, 'T': 1.20, 'V': 1.65, 'W': 1.19, 'Y': 1.29}

    cf = {'Ha': 1, 'ha': 1, 'Ia': 0.5, 'Ia': 0, 'ba': -1, 'Ba': -1,
          'Hb': 1, 'hb': 1, 'Ib': 0.5, 'Ib': 0, 'bb': -1, 'Bb': -1}
    #To store the final segments that are a part of either secondary structures.
    helix_seq = []
    strand_seq = []

    n = len(seq)
    #Identifying initial segments that fall into Alpha helices
    print('Alpha helices: ')
    i = 0
    while i<n-6:
        segment = seq[i:i+6]
        print(segment)
        value = 0
        for j in range(6):
            typ = helix[segment[j]]
            value+= cf[typ]
        print(value)
        if value>=4:
            done = 1
            k = 0
            while done==1:
                next_seg = seq[i+k+2:i+k+6]
                print(next_seg)
                prop = 0
                for l in range(4):
                    prop+=propensity_alpha[next_seg[l]]
                print(prop)
                if prop<4.00:
                    done = 0
                else:
                    k+=1
            if k==0:
                print(seq[i:i+k+6])
                helix_seq.append(seq[i:i+k+6])
                i = i+k+6
            else:
                print(seq[i:i+k+5])
                helix_seq.append(seq[i:i+k+5])
                i = i+k+5
        else:
            i+=1
    #Identifying initial segments that fall into Beta strands
    print('\nBeta strands: ')

```

```

print('\nBeta strands: ')
i = 0
while i < n-5:
    segment = seq[i:i+5]
    print(segment)
    value = 0
    for j in range(5):
        typ = strand[segment[j]]
        value += cf[typ]
    print(value)
    if value >= 3:
        done = 1
        k = 0

        while done == 1 and (i+k+5) <= n:
            next_seg = seq[i+k+2:i+k+5]
            print(next_seg)
            prop = 0
            for l in range(3):
                prop += propensity_beta[next_seg[l]]
            print(prop)
            if prop < 3.00:
                done = 0
            else:
                k += 1

        if k == 0:
            print(seq[i:i+k+5])
            strand_seq.append(seq[i:i+k+5])
            i = i+k+5
        else:
            print(seq[i:i+k+4])
            strand_seq.append(seq[i:i+k+4])
            i = i+k+4
    else:
        i += 1
#Lists to store conflicting segment's (wrong) index and the entire sequence exactly as where it belongs to
hf = []
sf = []
print('\nCommon segments - Conflicting sequences')
for i in range(len(helix_seq)):
    h = helix_seq[i]

    for j in range(len(strand_seq)):
        s = strand_seq[j]

        c = matchingString(h,s)
        m = len(c)
        if m != 0 and m >= 5:
            print('Helix - %s, Strand - %s, Common segment - %s' % (h,s,c))
            prop_helix = 0
            prop_strand = 0
            for k in range(m):
                prop_helix += propensity_alpha[c[k]]
                prop_strand += propensity_beta[c[k]]
            if prop_helix > prop_strand:
                hf.append([j,h])
            else:
                sf.append([i,s])

for i in range(len(hf)-1,-1,-1):
    a = strand_seq[hf[i][0]]
    strand_seq.remove(a)

for j in range(len(sf)-1,-1,-1):
    b = helix_seq[sf[j][0]]
    helix_seq.remove(b)

#Results
print('\nFinal list of secondary structure segments -')
print('\nAlpha Helix segments')
for i in range(len(helix_seq)):
    print(helix_seq[i])
print('\nBeta Strand segments')
for i in range(len(strand_seq)):
    print(strand_seq[i])

```

Alpha helices:

KVFGRC

1.5

VFGRCE

2

FGRCLE

2

GRCELA

2

RCELA

4

ELAA

5.7700000000000005

LAAA

5.69

AAAM

5.55

AAMK

5.17

AMKR

4.51

MKRH

4.3

KRHG

3.63

RCELAAAMKRH

GLDNYR

-2

LDNYRG

-2

DNYRGY

-4

NYRGYS

-4

YRGYSL

-2

RGYSLG

-2

GYSLGN

-3

YSLGNW

-1

SLGNWV

1

LGNWVC

1

GNWVCA

1

NWVCAA

3

WVCAAK

4.5

CAAK

4.74

AAKF

5.09

AKFE

5.17

KFES

4.5100000000000001

FESN

4.17

ESNF

4.17

SNFN

3.37

WVCAAKFESNF

NTQATN

0

TQATNR

1

QATNRN

0

ATNRNT

-1

TNRNTD

-2

NRNTDG

-3

RNTDGS

-2

NTDGST

-2

TDGSTD

-1

DGSTDY

-2

GSTDYG

-3

STDYGI

-1.5

TDYGIL

-0.5

DYGILQ

0.5

YGILQI

1.0

GILQIN

1.0

ILQINS

2.0

LQINSR

1.5

QINSRW

1.5

INSRWW

1.5

NSRWWC

1

SRWWCN

1

RWWCND

1

WWCNDG

0

WCNDGR

-1

CNDGRT

-2

NDGRTP

-3

DGRTPG

-3

GRTPGS

-3

RTPGSR

-2

TPGSRN

-3

PGSRNL

-2

GSRNLC

-1

SRNLCN

-1

RNLCNI

-0.5

NLCNIP

-1.5

LCNIPC

-0.5

CNIPCS

-1.5

NIPCSA

-0.5

IPCSAL

1.5

PCSALL

2

CSALLS

3

SALLSS

3

ALLSSD

3

LLSSDI

2.5

LSSDIT

1.5

SSDITA

1.5

SDITAS

1.5

DITASV

2.5

ITASVN

1.5

TASVNC

1

ASVNCA

2

SVNCAK

1.5

VNCAKK

2.0

NCAKKI

1.5

CAKKIV

3.5

AKKIVS

3.5

KKIVSD

2.5

KIVSDG

1.0

IVSDGN

-0.5

VSDGNG

-2

SDGNMG

-2

DGNGMN

-3

GNGMNA

-2

NGMNAW

0

GMNAWV

2

MNAWVA

4

AWVA

5.18

WVAW

4.8699999999999999

VAWR

4.52

AWRN

4.1099999999999999

WRNR

3.45

MNAWVAWRN

RCKGTD

-0.5

CKGTDV

0.5

KGTDVQ

1.5

GTDVQA

2

TDVQAW

4

VQAW

4.8999999999999995

QAWI

4.76

AWIR

4.38

WIRG

3.46

TDVQAWIR

Beta strands:

KVFGR

1

VFGRC

3

GRC

3.01

RCE

2.46

VFGRC

ELAAA

1.5

LAAAM

3.5

AAM

3.61

AMK

3.38

MKR

3.31

KRH

2.35

LAAAMKR

HGLDN

-1

GLDNY

1

LDNYR

1

DNYRG

0

NYRGY

1

YRGYS

1

RGYSL

1

GYSLG

1

YSLGN

0

SLGNW

0

LGNWV

2

GNWVC

2

NWVCA

2.5

WVCAA

4.0

CAA

3.24

AAK

2.6799999999999997

WVCAA

KFESN

-3

FESNF

-1

ESNFN

-3

SNFNT

-1

NFNTQ

1

FNTQA

2.5

NTQAT

2.5

TQATN

2.5

QATNR

1.5

ATNRN

-0.5

TNRNT

0

NRNTD

-1

RNTDG

0

NTDGS

-1

TDGST

1

DGSTD

0

GSTDY

1

STDYG

1

TDYGI

3

YGI

3.7

GIL

3.63

ILQ

4.0500000000000001

LQI

4.0500000000000001

QIN

3.48

INS

2.9699999999999998

TDYGILQIN

SRWWC

2

RWWCN

2

WWCND

2

WCNDG

1

CNDGR

0

NDGRT

0

DGRTP

0

GRTPG

0

RTPGS

-1

TPGSR

-1

PGSRN

-3

GSRNL

-1

SRNLC

0

RNLN

0

NLCNI

1

LCNIP

1

CNIPC

1

NIPCS

-1

IPCSA

0.5

PCSAL

0.5

CSALL

2.5

SALLS

0.5

ALLSS

0.5

LLSSD

0

LSSDI

0

SSDIT

0

SDITA

1.5

DITAS

1.5

ITASV

2.5

TASVN

0.5

ASVNC

0.5

SVNCA

0.5

VNCAK

0.5

NCAKK

-1.5

CAKKI

0.5

AKKIV

0.5

KKIVS

-1

KIVSD

0

IVSDG

1

VSDGN

-1

SDGNG

-2

DGNGM

0

GNGMN

-1

NGMNA

-0.5

GMNAW

1.5

MNAWV

2.5

NAWVA

2.0

AWVAW

4.0

VAW

3.81

AWR

3.06

WRN

2.7399999999999998

AWVAWR

NRCKG

-1

RCKGT

1

CKGTD

1

KGTDV

1

GTDVQ

3

DVQ

3.68

VQA

3.8499999999999996

QAW

3.39

AWI

3.7600000000000002

WIR

3.69

IRG

3.31

RGC

3.01

GCR

3.0100000000000002

CRL

3.42

GTDVQAWIRGCRL

Common segments - Conflicting sequences

Helix - RCELAAAMKRH, Strand - LAAAMKR, Common segment - LAAAMKR

Helix - WVCAAKFESNF, Strand - WVCAA, Common segment - WVCAA

Helix - MNAWVAWRN, Strand - AWVAVR, Common segment - AWVAVR

Helix - TDVQAWIR, Strand - GTDVQAWIRGCRL, Common segment - TDVQAWIR

Final list of secondary structure segments -

Alpha Helix segments

RCELAAAMKRH

MNAWVAWRN

Beta Strand segments

VFGRC

WVCAA

TDYGILQIN

GTDVQAWIRGCRL

Question 4

Helical Strands

Position (5-15) was classified helix

=> RCELAAMKRH

$$RCELAAM \Rightarrow 0 + 0 + 1 + 1 + 1 + 1 = 4 \text{ (using Chou - Frenkel parameters)}$$

Now, take last 4 incrementally till > 4

$$\Rightarrow ELAA \Rightarrow 1.53 + 1.34 + 1.45 + 1.45 = 5.77 > 4$$

$$LAAA \Rightarrow 1.34 + 1.75 + 1.45 + 1.45 = 5.69 > 4$$

$$AAAM \Rightarrow 1.45 + 1.75 + 1.75 + 1.20 = 5.55 > 4$$

$$AAMK \Rightarrow 1.45 + 1.75 + 1.20 + 1.07 = 5.17 > 4$$

$$AMKR \Rightarrow 1.45 + 1.20 + 1.07 + 0.79 = 4.51 > 4$$

$$MKRH \Rightarrow 1.20 + 1.07 + 0.79 + 1.24 = 4.3 > 4$$

$$KRHH \Rightarrow 1.07 + 0.79 + 1.24 + 0.53 = 3.63 < 4$$

Since KRHH gives < 4 , we terminate at MKRH

=> RCELAAMKRH is helical segment

CE+ending backward

$$GR(E) = 0.53 + 0.79 + 0.77 + 1.53$$

$$\Rightarrow 3.62 < 4$$

=> NO backward extension

Beta Strands

NWVCAA has been identified as beta strand

check 5 residue segments.

$$NWVCA \Rightarrow 0 + 1 + 1 + 1 + 0.5 = 3.5 \geq 3$$

Last 3 residues incrementally till > 3

$$VCA \Rightarrow 1.65 + 1.30 + 0.97 = 3.92 > 3$$

$$CAA \Rightarrow 1.30 + 0.97 + 0.97 = 3.24 > 3$$

$$AAK \Rightarrow 0.97 + 0.97 + 0.74 = 2.68 < 3$$

AAK gives < 3 , we terminate at CAA

$$GNW \Rightarrow 0.81 + 0.15 + 1.19 = 2.65 < 3$$

$\Rightarrow$ no extend backwards

$\Rightarrow$ NWVCAA is beta segment